

CRASH COURSE PLAN

Laravel Crash Course — REST API

| Lộ trình học chi tiết từng phần — kèm giải thích cơ chế hoạt động bên trong

Repo: [crash-course / laravel-crash-course](#)

Tài liệu bám theo **Laravel 13.x** (phát hành 17/03/2026, yêu cầu PHP 8.3–8.5) — kiểm chứng theo docs chính thức [laravel.com](#)

Kiểm tra phiên bản dự án của bạn bằng `php artisan --version`

Mục lục

PHP PHP 8.3+ cho người viết Laravel

NỀN TẢNG NGÔN NGỮ

00	Kiến trúc tổng thể & Request Lifecycle	NỀN TẢNG
01	Khởi tạo dự án & cấu trúc thư mục	
02	Routing & Controllers cho API	
03	Service Container, DI & Service Providers	
04	Database: Migration, Model (Eloquent), Factory & Seeder	
05	Validation với Form Request	
06	API Resources — tầng transform response	
07	Middleware & Exception Handling (chuẩn hoá lỗi JSON)	
08	Quan hệ dữ liệu (Relationships) & vấn đề N+1	
09	Authentication với Laravel Sanctum	
10	Chuẩn hoá REST API cho production	
11	Testing (Pest / PHPUnit)	TUỶ CHỌN

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh hoạ để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Dự án xuyên suốt: Xây một API quản lý bài viết — Post (tiêu đề, nội dung), Comment (bình luận thuộc về một Post), User (đăng ký / đăng nhập bằng token). Đây đúng là bài toán bạn đã làm bên NestJS, nên xuyên suốt tài liệu sẽ có các bảng **đối chiếu NestJS ↔ Laravel** để bạn thấy cùng một khái niệm được hai framework giải quyết thế nào.

PHP 8.3+ cho người viết Laravel

Mục tiêu: Nắm đủ PHP hiện đại để đọc/viết Laravel không bị "ngợp cú pháp". Phần này **bắt buộc** nếu bạn mới sang từ JS/TS — vì 90% khó khăn khi học Laravel thực chất là khó khăn với PHP, không phải với framework.

P.1 Mô hình thực thi: PHP khác Node.js ở chỗ căn bản nhất

Đặc điểm	Node.js / NestJS	PHP / Laravel (PHP-FPM)
Tiến trình	1 process sống mãi, xử lý mọi request	Mỗi request là một lần chạy script từ đầu , xong thì xoá sạch bộ nhớ
Bất đồng bộ	Event loop, mọi I/O đều async	Đồng bộ (blocking) theo mặc định — code chạy tuần tự từ trên xuống
Biến toàn cục / cache RAM	Giữ được giữa các request	Không giữ được — muốn cache phải dùng Redis/file/DB
Rò rỉ bộ nhớ	Nguy hiểm, tích tụ dần	Gần như không có — process chết sau mỗi request

Vì sao phải biết điều này: nó giải thích tại sao Laravel "singleton" chỉ sống trong 1 request (xem Part 03), tại sao Laravel không có `async/await` ở tầng controller, và tại sao code Laravel đọc rất "tuần tự" so với NestJS đầy Promise. (Laravel Octane thay đổi mô hình này, nhưng đó là chuyện sau.)

P.2 Cú pháp cốt lõi — đọc nhanh phần khác JS

```
<?php
declare(strict_types=1);           // NÊN có ở đầu mọi file: bật kiểm tra kiểu nghiêm ngặt

$name = 'Nhat';                    // biến LUÔN bắt đầu bằng $
const MAX = 10;                    // hằng số (không có $)

// Nháy đơn vs nháy kép — khác biệt quan trọng
echo 'Xin chào $name';             // → Xin chào $name   (KHÔNG nội suy)
echo "Xin chào $name";             // → Xin chào Nhat    (CÓ nội suy)
echo "Tổng: {$order->total}";       // nội suy biểu thức phức tạp thì bọc {}

// Nối chuỗi bằng dấu CHẤM, không phải dấu +
echo 'Xin ' . 'chào';              // 'Xin chào' ('a' + 'b' là LỖI trong PHP 8

// So sánh: LUÔN dùng === (== có ép kiểu ngầm, nguồn bug kinh điển)
0 == '0';                          // true
0 === '0';                         // false ← dùng cái này
```

P.3 Toán tử hiện đại (PHP 8) — Laravel dùng liên tục

Cú pháp	Ý nghĩa	Tương đương JS/TS
<code>\$a ?? 'default'</code>	Null coalescing: nếu null/không tồn tại thì lấy default	<code>a ?? 'default'</code>
<code>\$a ??= 5</code>	Gán nếu đang null	<code>a ??= 5</code>
<code>\$user?->profile?->city</code>	Nullsafe: null thì dừng, không lỗi	<code>user?.profile?.city</code>
<code>\$a <=> \$b</code>	Spaceship: trả -1 / 0 / 1 — dùng cho sort	hàm compare
<code>...\$items</code>	Spread / variadic	<code>...items</code>
<code>fn (\$x) => \$x * 2</code>	Arrow function (tự bắt biến ngoài)	<code>(x) => x * 2</code>

```
// match (PHP 8) — thay thế switch, so sánh === và TRẢ VỀ giá trị
$label = match($status) {
    200, 201 => 'success',
    404      => 'not found',
    default  => 'unknown',      // thiếu default + không khớp → throw UnhandledMatchError
};

// match(true) — mẹo rất hay dùng trong Laravel
$tier = match(true) {
    $total > 1000 => 'vip',
    $total > 100  => 'normal',
    default      => 'new',
};
```

P.4 Array — chỉ có MỘT kiểu, gán cả object của JS

```
// Indexed array (như Array của JS)
$tags = ['php', 'laravel'];

// Associative array (như Object/Map của JS) — RẤT hay gặp trong Laravel
$post = ['title' => 'Hello', 'views' => 10];
echo $post['title']; // truy cập bằng [] chứ không phải .

foreach ($post as $key => $value) { /* ... */ }

// Các hàm array phải thuộc lòng
array_map(fn ($p) => $p['title'], $posts); // map
array_filter($posts, fn ($p) => $p['views'] > 5); // filter (GIỮ NGUYÊN key!)
array_reduce($posts, fn ($c, $p) => $c + $p['views'], 0);
array_column($posts, 'title'); // lấy 1 cột từ mảng 2 chiều
array_keys($post); array_values($post); count($post);
in_array('php', $tags, true); // tham số 3 = so sánh strict
```

Bẫy: array_map đảo thứ tự tham số so với array_filter (callback trước / mảng trước), và array_filter giữ nguyên key cũ nên mảng kết quả có thể "khuyết index" — khi trả JSON sẽ thành object thay vì array. Dùng array_values(...) để reset index. Trong Laravel bạn sẽ ít gặp chuyện này vì **Collection** (collect(\$arr) -> map() -> filter() -> values()) xử lý gọn hơn nhiều — nhưng phải biết array thuần trước.

P.5 Function & type declaration

```
// Khai kiểu đầy đủ – bắt buộc nếu muốn code Laravel chuyên nghiệp
function slugify(string $title, int $maxLength = 60): string
{
    return substr(strtolower(str_replace(' ', '-', $title)), 0, $maxLength);
}

function find(int $id): ?Post { }          // ?Post = Post hoặc null
function parse(string|int $v): array { }   // union type
function log(string ...$lines): void { }   // variadic: nhận n tham số

// Named arguments (PHP 8) – Laravel dùng nhiều, vd apiPrefix: 'api/v1'
slugify(title: 'Hello World', maxLength: 20);

// Closure vs arrow fn: closure PHẢI khai báo use() để lấy biến ngoài
$rate = 1.1;
$f1 = function ($x) use ($rate) { return $x * $rate; }; // tường minh
$f2 = fn ($x) => $x * $rate;                             // tự bắt, gọn hơn
```

P.6 OOP — phần Laravel dùng nhiều nhất

```
namespace App\Services;                    // namespace ~ đường dẫn thư mục

use App\Models\Post;                       // import class (giống import của JS)

class PostService
{
    // Constructor property promotion (PHP 8): khai báo + gán trong 1 dòng
    public function __construct(
        private readonly Repository $repo,    // readonly: gán 1 lần, không sửa được
        protected int $perPage = 15,
    ) {}

    public function latest(): array
    {
        return $this->repo->all();              // $this-> (không phải this.)
    }

    public static function make(): static
    {
        return new static();                  // self:: / static:: cho thành viên static
    }
}
```

Khái niệm	Ý nghĩa & nơi Laravel dùng
<code>interface</code>	Hợp đồng (chỉ khai method, không có thân). Laravel gọi là Contract — vd <code>Illuminate\Contracts\Auth\Authenticatable</code>
<code>abstract class</code>	Class không tạo instance trực tiếp, để kế thừa — vd <code>Model</code> , <code>FormRequest</code>
<code>trait</code>	Ghép code vào class (PHP không đa kế thừa). Laravel dùng khắp nơi: <code>HasApiTokens</code> , <code>SoftDeletes</code> , <code>HasFactory</code> , <code>RefreshDatabase</code>
<code>enum</code> (PHP 8.1)	Tập giá trị cố định — rất nên dùng cho status. Cast được thẳng vào Eloquent (Part 04)
<code>static</code> vs <code>self</code>	<code>static</code> = late static binding (theo class con), <code>self</code> = cố định class hiện tại

```
// Trait – hiểu nó là hiểu "phép thuật" của Laravel
trait Sluggable
{
    public function slug(): string { return slugify($this->title); }
}

class Post extends Model
{
    use Sluggable, HasFactory; // "dán" method của trait vào class này
}

// Enum backed – dùng cho trạng thái
enum PostStatus: string
{
    case Draft = 'draft';
    case Published = 'published';

    public function label(): string
    {
        return match($this) {
            self::Draft => 'Bản nháp',
            self::Published => 'Đã xuất bản',
        };
    }
}
```

P.7 Attribute (PHP 8) — cú pháp giống decorator của TS

```
// Attribute là METADATA gắn lên class/method/property, đọc lại bằng Reflection
#[Fillable(['title', 'body'])]
#[Table('posts')]
class Post extends Model { }
```

Điểm khác decorator của TypeScript/NestJS: attribute của PHP **hoàn toàn thụ động** — nó không tự chạy code, chỉ ngời đó cho tới khi có ai chủ động đọc nó bằng Reflection. Laravel 13 chuyển nhiều cấu hình model sang attribute (`#[Fillable]`, `#[Table]`, `#[Scope]`, `#[RouteKey]`) và thêm attribute cho controller (`#[Middleware]`, `#[Authorize]`) — nên bạn sẽ gặp cú pháp này liên tục.

P.8 Namespace, Composer & autoload PSR-4

```
// composer.json – phần quan trọng nhất
{
    "require": { "php": "^8.3", "laravel/framework": "^13.0" },
    "require-dev": { "pestphp/pest": "^4.0" },
    "autoload": { "psr-4": { "App\\": "app/" } }
}
```

PSR-4 là quy ước: namespace phải phản chiếu đúng đường dẫn thư mục. `App\Services\PostService` ⇒ file `app/Services/PostService.php`. Sai một chữ (kể cả sai hoa/thường) là "class not found". Không cần require file thủ công — `vendor/autoload.php` tự nạp khi bạn use.

Lệnh Composer	Tác dụng
<code>composer install</code>	Cài đúng version trong <code>composer.lock</code> (dùng khi deploy / clone repo)
<code>composer update</code>	Nâng version theo constraint + ghi lại lock (cẩn thận, đừng chạy khi deploy)
<code>composer require vendor/pkg</code>	Thêm package mới
<code>composer dump-autoload</code>	Nạp lại map autoload khi thêm class mà báo "not found"

P.9 Exception

```
try {
    $post = $service->publish($id);
} catch (ModelNotFoundException $e) {
    // bắt loại cụ thể trước
} catch (\Throwable $e) {
    // Throwable = cha của MỌI lỗi (Exception + Error). Bắt cuối cùng.
    report($e); // helper Laravel: ghi log rồi tiếp tục
} finally {
    // luôn chạy
}

throw new \InvalidArgumentException('ID phải > 0');
```

Trong Laravel bạn sẽ rất ít viết try/catch. Triết lý là: cứ throw, để Exception Handler tập trung lo việc biến nó thành response (Part 07). Try/catch chỉ dùng khi bạn **thực sự xử lý được** lỗi tại chỗ đó.

P.10 Bảng tra nhanh PHP ↔ TypeScript

TypeScript / NestJS	PHP / Laravel
<code>const x = 1</code>	<code>\$x = 1;</code> (biến), <code>const X = 1;</code> (hằng)
<code>this.foo</code>	<code>\$this->foo</code>
<code>obj.prop</code>	<code>\$obj->prop</code> (object) / <code>\$arr['key']</code> (array)
<code>Class.staticMethod()</code>	<code>Class::staticMethod()</code>
<code>`text \${v}`</code>	<code>"text \$v"</code> hoặc <code>'text ' . \$v</code>
<code>arr.map(f)</code>	<code>array_map(\$f, \$arr)</code> / <code>collect(\$arr)->map(\$f)</code>
<code>interface</code> + <code>implements</code>	Giống hệt (PHP có <code>interface</code> , <code>implements</code>)
Mixin / kế thừa nhiều	<code>trait</code>
<code>@Decorator()</code>	<code>#[Attribute]</code> (thụ động, cần Reflection đọc)
<code>async</code> / <code>await</code>	Không có ở tầng app — code chạy đồng bộ
<code>npm</code> / <code>package.json</code>	<code>composer</code> / <code>composer.json</code>
<code>eslint</code> + <code>prettier</code>	<code>Laravel Pint</code> (chuẩn PSR-12) — chạy <code>./vendor/bin/pint</code>

Việc cần làm (làm bằng PHP thuần, chưa cần Laravel):

1. Tạo thư mục `php-warmup/`, viết file `index.php` có `declare(strict_types=1)`, chạy bằng `php index.php`
2. Viết hàm `slugify(string $title): string` có khai báo kiểu đầy đủ; thử truyền số vào để xem `strict_types` báo lỗi thế nào
3. Tạo mảng 5 post (associative array), dùng `array_filter` + `array_map` + `array_values` để lấy tiêu đề các post có views > 10, in ra bằng `json_encode`
4. Viết class `Post` + interface `Publishable` + trait `Sluggable` + enum `PostStatus: string` — dùng constructor property promotion và `readonly`
5. Tạo `composer.json` với autoload PSR-4 `"App\\" : "app/"`, đặt class vào `app/`, chạy `composer dump-autoload` rồi use class đó từ `index.php`
6. Viết một exception tự định nghĩa và `throw` / `catch` nó
7. Trả lời được: vì sao PHP không giữ được biến giữa 2 request, và điều đó ảnh hưởng gì tới cách bạn cache dữ liệu

Kiến trúc tổng thể & Request Lifecycle

Đây là phần quan trọng nhất, cần đọc kỹ trước khi code. Laravel không phải "một đồng helper function" — nó là một **IoC container** (kho chứa và tự lắp ráp object) được bootstrap bởi các **Service Provider**, rồi đẩy request qua một **pipeline middleware**. Hiểu 3 khái niệm đó là hiểu 80% Laravel.

1. Điểm khởi đầu: `public/index.php` → `bootstrap/app.php`

Mọi request HTTP đều được web server (Nginx/Apache) trở vào đúng một file duy nhất: `public/index.php`. File này rất ngắn, nó chỉ làm 3 việc: nạp Composer autoloader, lấy instance ứng dụng từ `bootstrap/app.php`, rồi gọi `handleRequest()`.

```
// bootstrap/app.php – "file cấu hình trung tâm" của Laravel 11+
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',    // chỉ có sau khi chạy install:api
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        // đăng ký / sắp xếp middleware ở đây
    })
    ->withExceptions(function (Exceptions $exceptions) {
        // tùy biến cách báo lỗi & render lỗi ở đây
    })->create();
```

Điểm khác biệt lớn so với Laravel 10 trở về trước: từ Laravel 11, các file `app/Http/Kernel.php`, `app/Exceptions/Handler.php`, `app/Providers/RouteServiceProvider.php` đã bị **xoá**. Toàn bộ cấu hình middleware, exception, routing gom hết về `bootstrap/app.php`. Nếu bạn xem tutorial cũ trên YouTube/blog mà thấy sửa `Kernel.php` → đó là tài liệu lỗi thời, đừng làm theo.

2. Service Container (IoC Container) — trái tim của Laravel

Service Container là một "kho" biết cách **tự khởi tạo object** và **tự tiêm dependency**. Khi bạn type-hint một class ở constructor (hoặc ở tham số method của controller), Laravel dùng **PHP Reflection** để đọc kiểu dữ liệu đó, tự tạo instance rồi truyền vào — bạn không bao giờ phải tự `new PostService()`.

```
class PostController extends Controller
{
    public function __construct(
        protected PostService $posts,    // container tự resolve và tiêm vào
    ) {}
}
```

Đây chính xác là cơ chế Dependency Injection bạn đã gặp ở NestJS. Khác biệt: NestJS đọc metadata do decorator sinh ra lúc compile; Laravel đọc trực tiếp type-hint qua Reflection lúc runtime — **không cần đăng ký class vào một mảng providers** nào cả nếu class đó có thể tự khởi tạo được.

3. Service Providers — nơi mọi thứ được "lắp ráp"

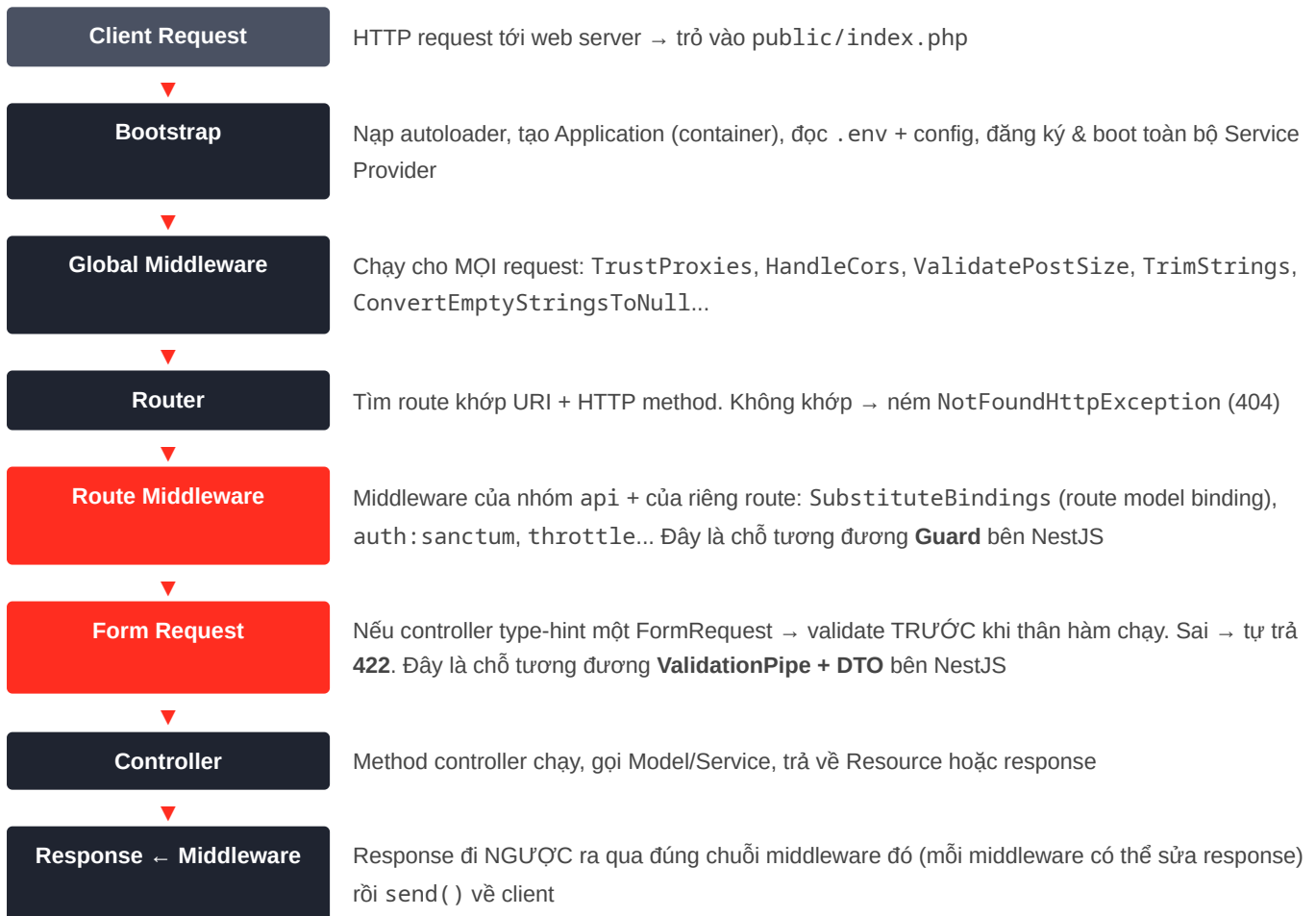
Service Provider là các class chịu trách nhiệm khai báo cho container biết: "gặp interface X thì dựng class Y", "service Z là singleton"... Mỗi provider có 2 method, và **thứ tự chạy của chúng rất quan trọng**:

Method	Chạy khi nào	Được làm gì
<code>register()</code>	Chạy trước , trên TẤT CẢ provider	Chỉ được đăng ký binding vào container. KHÔNG được dùng service của provider khác (vì có thể chưa đăng ký xong)
<code>boot()</code>	Chạy sau , khi mọi provider đã register xong	Làm mọi thứ cần dùng tới service khác: định nghĩa rate limiter, route model binding, validation rule, event listener...

Danh sách provider của ứng dụng nằm ở `bootstrap/providers.php`; provider mặc định của bạn là `app/Providers/AppServiceProvider.php`.

4. Request Lifecycle — thứ tự một request đi qua hệ thống

Đây là sơ đồ cần thuộc. So với NestJS (Middleware → Guard → Interceptor → Pipe → Handler), Laravel gom gọn hơn: **mọi thứ chặn/lọc request đều là middleware**, còn validate dữ liệu thì nằm ở Form Request.



⚠ Bất kỳ bước nào **throw exception** → nhảy thẳng tới Exception Handler (cấu hình trong `withExceptions`) → render thành response lỗi. Với request có header `Accept: application/json`, Laravel tự trả lỗi dạng JSON.

5. Facade thực chất là gì?

Khi bạn viết `Route::get(...)` hay `DB::table(...)`, trông như gọi static method, nhưng thực ra **không có static method nào cả**. Facade là một class proxy: nó bắt lời gọi static bằng `__callStatic()`, hỏi container lấy

instance thật đang được đăng ký, rồi gọi method **instance** trên đó. Nghĩa là Facade chỉ là "lối tắt cú pháp" để lấy đồ trong container — object bên dưới vẫn mock được khi test.

6. Đối chiếu nhanh NestJS ↔ Laravel

NestJS	Laravel	Ghi chú
Module (@Module)	Service Provider	Laravel không đóng gói theo module; code tổ chức theo thư mục + provider
IoC container + providers: []	Service Container (tự resolve qua Reflection)	Laravel không cần khai báo trước nếu class tự dựng được
Controller + @Get()	Controller + routes/api.php	Laravel tách route ra file riêng thay vì gắn decorator
DTO + class-validator	Form Request	Cùng vai trò: validate input trước khi vào handler
Guard (CanActivate)	Middleware (auth:sanctum)	Laravel gộp Guard vào middleware
Interceptor (transform response)	API Resource	Laravel transform ở tầng Resource thay vì bọc response
Exception Filter	withExceptions() trong bootstrap/app.php	Cùng mục đích: chuẩn hoá format lỗi
TypeORM Repository (Data Mapper)	Eloquent Model (Active Record)	Eloquent: bản thân model tự biết lưu/xoá chính nó

Việc cần làm:

- Chưa code gì cả — đọc hiểu sơ đồ lifecycle ở mục 4, tự vẽ lại ra giấy
- Trả lời được: khác nhau giữa register() và boot() là gì, và tại sao phải tách làm 2 bước
- Trả lời được: Facade khác gì với static method thật

Khởi tạo dự án & cấu trúc thư mục

Mục tiêu: Dựng được project Laravel chạy được, hiểu mỗi thư mục dùng để làm gì, và bật chế độ API (điều mà Laravel **không** bật sẵn).

1.1 Cài đặt

Yêu cầu: **PHP 8.3 trở lên** (Laravel 13 hỗ trợ PHP 8.3–8.5) và **Composer**. Kiểm tra bằng `php -v` và `composer -V`.

```
# Cách 1 – dùng Composer (không cần cài thêm gì)
composer create-project laravel/laravel laravel-crash-course

# Cách 2 – dùng Laravel installer (cài 1 lần, dùng mãi)
composer global require laravel/installer
laravel new laravel-crash-course

# Chạy dev server
cd laravel-crash-course
php artisan serve          # → http://127.0.0.1:8000
```

Khi cài sẽ có bước hỏi chọn starter kit / testing framework. Với crash course API này, chọn **không dùng starter kit** (none) vì ta không làm giao diện. Phần test framework chọn **Pest** (mặc định) — sẽ dùng ở Part 11.

1.2 Bật API — bước bắt buộc, rất nhiều người quên

Laravel 11+ **không tạo sẵn file routes/api.php**. Dự án mới chỉ có routes/web.php. Muốn làm API bạn phải chạy:

```
php artisan install:api
```

Lệnh này làm 3 việc: (1) cài **Laravel Sanctum** (authentication bằng API token — học ở Part 09), (2) tạo file `routes/api.php`, (3) thêm dòng `api:` vào `withRouting()` trong `bootstrap/app.php` đồng thời publish migration bằng `personal_access_tokens`.

Hệ quả quan trọng: mọi route khai trong `routes/api.php` được **tự động thêm prefix /api** và gán nhóm middleware `api`. Khai Route: `::get('/posts', ...)` → URL thật là **/api/posts**. Đừng tự thêm `/api` vào route nữa, sẽ thành `/api/api/posts`.

1.3 Cấu hình môi trường (.env)

File `.env` chứa config theo từng máy (DB, key, debug) và **không bao giờ được commit lên git** (đã có sẵn trong `.gitignore`). File `.env.example` mới là file được commit — nó là "bản kê khai" các biến cần có để người khác biết đường tự tạo `.env` của họ.

```

APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:...      # sinh bởi php artisan key:generate — dùng để mã hoá cookie/session
APP_DEBUG=true          # PRODUCTION BẮT BUỘC = false (nếu true sẽ lộ config, stack trace)
APP_URL=http://localhost

DB_CONNECTION=sqlite    # mặc định của Laravel 11+ — nhẹ, không cần cài server
# Muốn dùng MySQL thì đổi thành:
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1 / DB_PORT=3306 / DB_DATABASE=... / DB_USERNAME=... / DB_PASSWORD=...

```

Quy tắc vàng: KHÔNG gọi `env('APP_DEBUG')` rải rác trong code. Chỉ được gọi `env()` bên trong các file `config/*.php`, còn code ứng dụng thì đọc qua `config('app.debug')`. Lý do: khi chạy `php artisan config:cache` ở production, mọi lời gọi `env()` ngoài config sẽ trả về **null** — đây là lỗi kinh điển làm sập app khi deploy.

1.4 Cấu trúc thư mục cần nhớ

Đường dẫn	Vai trò
<code>public/index.php</code>	Entry point duy nhất — web server phải trỏ document root vào thư mục <code>public/</code>
<code>bootstrap/app.php</code>	Cấu hình trung tâm: routing, middleware, exception (Laravel 11+)
<code>bootstrap/providers.php</code>	Danh sách Service Provider của ứng dụng
<code>routes/api.php</code>	Route API (prefix <code>/api</code> , middleware group <code>api</code>)
<code>routes/web.php</code>	Route web (có session + CSRF) — API không dùng tới
<code>app/Http/Controllers/</code>	Controller
<code>app/Http/Requests/</code>	Form Request — nơi validate input (Part 05)
<code>app/Http/Resources/</code>	API Resource — nơi định dạng output (Part 06)
<code>app/Models/</code>	Model Eloquent
<code>app/Providers/</code>	Service Provider của bạn
<code>database/migrations/</code>	Lịch sử thay đổi cấu trúc bảng
<code>database/factories/</code> , <code>seeders/</code>	Sinh dữ liệu giả để dev & test
<code>config/</code>	File cấu hình (Laravel 11+ rút gọn, chạy <code>php artisan config:publish</code> để lấy thêm)
<code>storage/logs/laravel.log</code>	Log — chỗ đầu tiên phải xem khi có lỗi 500

1.5 Vài lệnh artisan sống còn

Lệnh	Dùng để
<code>php artisan list</code>	Xem toàn bộ lệnh có sẵn
<code>php artisan route:list --path=api</code>	Liệt kê route API đã đăng ký (kiểm tra nhanh nhất)
<code>php artisan make:...</code>	Sinh class (controller, model, request, resource...)
<code>php artisan migrate</code>	Chạy migration lên database
<code>php artisan tink</code>	REPL — gõ thẳng code PHP có sẵn toàn bộ app (test Eloquent cực nhanh)
<code>php artisan optimize:clear</code>	Xoá mọi cache (config/route/view) khi thấy "sửa mà không ăn"

Việc cần làm:

1. Tạo project `laravel-crash-course` trong repo `crash-course`
2. Chạy `php artisan install:api`, xác nhận file `routes/api.php` đã xuất hiện và `bootstrap/app.php` có thêm dòng `api`:
3. Chạy `php artisan serve`, mở `http://127.0.0.1:8000/up` (health check) — phải thấy trang báo `application up`
4. Chạy `php artisan route:list` và đọc hiểu output: cột `Method` / `URI` / `Action` / `Middleware`
5. Mở `bootstrap/app.php` đọc kỹ 4 method `withRouting` / `withMiddleware` / `withExceptions` / `create`

Routing & Controllers cho API

Mục tiêu: Map được HTTP request tới đúng method trong Controller theo chuẩn REST, và hiểu Route Model Binding — thứ giúp bạn xóa sạch code "tìm bản ghi, không thấy thì trả 404".

2.1 Khai báo route

```
// routes/api.php — nhớ: đã tự có prefix /api
use App\Http\Controllers\PostController;

Route::get('/posts', [PostController::class, 'index']);
Route::post('/posts', [PostController::class, 'store']);
Route::get('/posts/{post}', [PostController::class, 'show']);
Route::put('/posts/{post}', [PostController::class, 'update']);
Route::delete('/posts/{post}', [PostController::class, 'destroy']);
```

2.2 apiResource — viết 5 route trên bằng 1 dòng

```
Route::apiResource('posts', PostController::class);
```

Verb	URI	Method	Route name
GET	/api/posts	index	posts.index
POST	/api/posts	store	posts.store
GET	/api/posts/{post}	show	posts.show
PUT/PATCH	/api/posts/{post}	update	posts.update
DELETE	/api/posts/{post}	destroy	posts.destroy

`apiResource` khác `resource` ở chỗ nó bỏ 2 route **create** và **edit** — hai route đó chỉ để trả về form HTML, API thuần không cần. Sinh controller tương ứng bằng:

```
# --api: sinh sẵn 5 method (không có create/edit)
# --model: type-hint sẵn model Post vào tham số (bật route model binding)
# --requests: sinh luôn StorePostRequest + UpdatePostRequest
php artisan make:controller PostController --api --model=Post --requests
```

PUT hay PATCH? Chuẩn HTTP: **PUT** = thay thế toàn bộ resource (client gửi đủ mọi field), **PATCH** = sửa một phần (chỉ gửi field cần đổi). `apiResource` đăng ký cả hai cùng trở vào `update()`, nên bạn tự quyết định: nếu rule validation cho phép thiếu field (sometimes) thì thực chất bạn đang làm PATCH — hãy để client gọi PATCH cho đúng ngữ nghĩa.

2.3 Route Model Binding — tính năng "ăn tiền" nhất

Thay vì nhận `$id` rồi tự query, bạn type-hint thẳng Model. Middleware `SubstituteBindings` (có sẵn trong nhóm `api`) sẽ tự query bản ghi theo giá trị URI, và tự trả 404 nếu không tìm thấy:

```
// Route: /api/posts/{post} → tên tham số {post} phải TRÙNG tên biến $post
public function show(Post $post)
{
    return $post;    // tới được đây nghĩa là chắc chắn tìm thấy
}
```

So sánh với NestJS: bên đó bạn phải viết `parseIntPipe` + tự `throw new NotFoundException()` trong service. Laravel làm sẵn cả hai.

Muốn tìm theo cột khác `id` (ví dụ `slug`) — có 2 cách:

```
// Cách 1: khai ngay tại route (chỉ áp dụng cho route đó)
Route::get('/posts/{post:slug}', [PostController::class, 'show']);

// Cách 2 (Laravel 13): gắn attribute lên Model → áp dụng cho MỌI route bind Post
use Illuminate\Database\Eloquent\Attributes\RouteKey;

#[RouteKey('slug')]
class Post extends Model { }
```

2.4 Controller gọn gàng

```
namespace App\Http\Controllers;

use App\Models\Post;

class PostController extends Controller
{
    public function index()
    {
        return Post::latest()->paginate(15);
    }

    public function store(StorePostRequest $request)    // validate tự chạy trước
    {
        $post = Post::create($request->validated());

        return response()->json($post, 201);    // 201 Created
    }

    public function destroy(Post $post)
    {
        $post->delete();

        return response()->noContent();    // 204 No Content
    }
}
```

Nguyên tắc "controller mỏng": controller chỉ nên làm 3 việc — nhận request đã validate, gọi model/service, trả response. Không đặt logic nghiệp vụ phức tạp, không viết query dài dòng ở đây.

2.5 Nhóm route & middleware

```
Route::apiResource('posts', PostController::class)->only(['index', 'show']);

// Nhóm route cần đăng nhập
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('posts', PostController::class)->except(['index', 'show']);
});

// Hoặc chỉ định theo từng action (Laravel 12+)
Route::apiResource('posts', PostController::class)
    ->middlewareFor(['store', 'update', 'destroy'], 'auth:sanctum');
```

Việc cần làm:

1. Tạo PostController bằng `make:controller --api`
2. Đăng ký `Route::apiResource('posts', ...)` trong `routes/api.php`
3. Tạm thời trả về mảng cứng trong bộ nhớ (chưa cần database) cho đủ 5 action
4. Chạy `php artisan route:list --path=api`, đối chiếu với bảng ở mục 2.2
5. Test 5 endpoint bằng Postman/curl. **Nhớ gửi header `Accept: application/json`** — thiếu header này Laravel sẽ trả HTML khi có lỗi
6. Thử gọi `/api/posts/999` và quan sát response khi chưa có model binding

Service Container, DI & Service Providers

Mục tiêu: Tách logic nghiệp vụ ra khỏi Controller, và hiểu container tự lắp ráp object như thế nào — nền tảng để code test được.

3.1 Tự động resolve (zero config)

Container đọc type-hint bằng Reflection và tự dựng object. Với class không có dependency đặc biệt, bạn **không** cần khai báo gì cả:

```
namespace App\Services;

class PostService
{
    public function publish(Post $post): Post
    {
        $post->update(['published_at' => now()]);

        return $post;
    }
}

// Controller – chỉ cần type-hint, container lo phần còn lại
class PostController extends Controller
{
    public function __construct(protected PostService $posts) {}
}
```

Khác NestJS ở đây: Nest bắt buộc khai providers: [PostService] trong module, quên là lỗi "Nest can't resolve dependencies". Laravel không cần — chỉ khi bạn muốn **can thiệp** vào cách dựng object mới phải đăng ký binding.

3.2 Khi nào phải bind thủ công

Khi type-hint là **interface** (container không biết dựng interface), hoặc khi cần truyền tham số đặc biệt, hoặc muốn dùng chung một instance:

```
// app/Providers/AppServiceProvider.php
public function register(): void
{
    // bind: mỗi lần resolve tạo instance MỚI (giống scope TRANSIENT bên Nest)
    $this->app->bind(PaymentGateway::class, StripeGateway::class);

    // singleton: tạo 1 lần, dùng lại mãi (giống scope DEFAULT bên Nest)
    $this->app->singleton(PostService::class, function ($app) {
        return new PostService(config('services.posts.per_page'));
    });

    // scoped: 1 instance cho mỗi request (quan trọng khi chạy Octane)
    $this->app->scoped(RequestContext::class, fn () => new RequestContext());
}
```

Cách bind	Vòng đời instance	Tương đương NestJS
bind	Tạo mới mỗi lần được resolve	Scope.TRANSIENT
singleton	Một instance duy nhất cho cả app	Scope.DEFAULT
scoped	Một instance cho mỗi request	Scope.REQUEST

Lưu ý về vòng đời process: PHP-FPM mặc định **huỷ toàn bộ bộ nhớ sau mỗi request** — nên "singleton" của Laravel thực chất chỉ sống trong một request, khác hẳn Node.js/NestJS nơi singleton sống suốt đời tiến trình server. Đây là lý do bạn không thể "cache dữ liệu vào thuộc tính static" trong Laravel như bên Nest.

3.3 Bind theo interface — vì sao nên làm

```
// Contract
interface NotifierInterface { public function send(string $msg): void; }

// Ở AppServiceProvider::register()
$this->app->bind(NotifierInterface::class, MailNotifier::class);
```

Lợi ích: đổi MailNotifier → SlackNotifier chỉ sửa 1 dòng; khi test thì bind sang một fake object. Đây là nguyên tắc **Dependency Inversion** — code phụ thuộc vào abstraction, không phụ thuộc implementation cụ thể.

3.4 register() vs boot() — lỗi hay gặp

```
public function register(): void
{
    // ✓ CHỈ đăng ký binding ở đây
    $this->app->singleton(PostService::class);
    // ✗ SAI: Route::pattern(...) / RateLimiter::for(...) — lúc này provider khác có thể chưa sẵn sàng
}

public function boot(): void
{
    // ✓ Mọi thứ cần dùng service khác thì đặt ở đây
    RateLimiter::for('api', fn ($request) =>
        Limit::perMinute(60)->by($request->user()?->id ?: $request->ip()));
}
```

Việc cần làm:

1. Tạo `app/Services/PostService.php`, chuyển toàn bộ logic từ Controller sang đây
2. Inject `PostService` vào Controller qua constructor — xác nhận chạy được mà KHÔNG cần đăng ký gì
3. Tạo interface `PostRepositoryInterface` + một implementation, bind trong `AppServiceProvider::register()`, rồi type-hint interface đó ở service
4. Thử cố tình bind sai (bind interface tới class không tồn tại) để xem thông báo lỗi của container trông thế nào
5. Trả lời: vì sao `RateLimiter::for()` phải nằm trong `boot()` chứ không phải `register()`

Database: Migration, Eloquent Model, Factory & Seeder

Mục tiêu: Thay mảng cứng bằng database thật, hiểu Eloquent (Active Record) khác gì TypeORM Repository (Data Mapper), và biết bảo vệ ứng dụng khỏi lỗi hỏng mass assignment.

4.1 Migration — "git cho cấu trúc database"

Migration là file PHP mô tả một thay đổi cấu trúc bảng, chạy theo thứ tự thời gian. Nhờ nó, đồng đội chỉ cần git pull + php artisan migrate là có database giống hệt bạn.

```
php artisan make:migration create_posts_table
```

```
public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id(); // BIGINT UNSIGNED AUTO_INCREMENT PK
        $table->string('title');
        $table->text('body');
        $table->timestamp('published_at')->nullable();
        $table->timestamps(); // tạo created_at + updated_at
    });
}

public function down(): void
{
    Schema::dropIfExists('posts'); // dùng khi rollback
}
```

Lệnh	Tác dụng
<code>php artisan migrate</code>	Chạy các migration chưa chạy
<code>php artisan migrate:rollback</code>	Lùi lại batch migration gần nhất
<code>php artisan migrate:fresh --seed</code>	Xoá sạch mọi bảng, chạy lại từ đầu + seed. Chỉ dùng ở local
<code>php artisan migrate:status</code>	Xem migration nào đã chạy

Nguyên tắc: migration đã chạy trên production thì **không sửa lại file cũ** — phải tạo migration mới để thay đổi. Sửa file cũ sẽ khiến máy người khác (đã chạy rồi) không bao giờ nhận được thay đổi đó.

4.2 Model Eloquent & quy ước ngầm

```
php artisan make:model Post -mf # -m: kèm migration, -f: kèm factory
php artisan make:model Post -a # -a: kèm migration+factory+seeder+policy+controller+requests
```

Quy ước	Mặc định Eloquent hiểu là
Tên bảng	snake_case số nhiều của tên class: <code>Post</code> → <code>posts</code>
Khoá chính	Cột <code>id</code> , kiểu số tự tăng
Timestamps	Tự quản lý <code>created_at</code> / <code>updated_at</code>
Khoá ngoại	<code>Comment</code> thuộc <code>Post</code> → cột <code>post_id</code>

Muốn đổi quy ước, Laravel 13 dùng **PHP attribute** thay cho các thuộc tính protected kiểu cũ:

```
use Illuminate\Database\Eloquent\Attributes\Table;

#[Table('blog_posts', key: 'post_id')]
class Post extends Model { }
```

4.3 Mass assignment — bảo mật, không phải thủ tục

Muốn dùng `Post::create($data)`, bạn **phải** khai báo field nào được phép gán hàng loạt. Đây là cơ chế chống việc kẻ tấn công gửi thêm field lạ (ví dụ `is_admin=1, user_id=999`) rồi ghi thẳng vào DB.

```
use Illuminate\Database\Eloquent\Attributes\Fillable;

#[Fillable(['title', 'body', 'published_at'])] // cú pháp Laravel 13
class Post extends Model
{
    protected function casts(): array // ép kiểu khi đọc/ghi
    {
        return [
            'published_at' => 'datetime', // string trong DB → Carbon object trong PHP
            'is_featured' => 'boolean', // 0/1 → true/false
        ];
    }
}
```

Cú pháp cũ vẫn chạy: `protected $fillable = ['title', 'body'];` vẫn hoạt động bình thường và bạn sẽ gặp nó ở 99% tutorial trên mạng. Attribute `#[Fillable]` là cách viết mới mà docs Laravel 13 dùng — chọn **một** kiểu và dùng nhất quán trong cả project.

4.4 CRUD với Eloquent

```
// ĐỌC
Post::all(); // mọi bản ghi – cẩn thận với bảng lớn
Post::find(1); // không thấy → null
Post::findOrFail(1); // không thấy → ModelNotFoundException → tự thành 404
Post::where('title', 'like', '%nest%')->get();
Post::latest()->paginate(15); // phân trang, kèm sẵn meta/links

// TẠO
$post = Post::create(['title' => 'Hello', 'body' => '...']);

// SỬA
$post->update(['title' => 'Updated']);

// XOÁ
$post->delete();
```

Active Record vs Data Mapper: Eloquent là **Active Record** — bản thân object `$post` biết cách tự lưu/xoá chính nó (`$post->save()`). TypeORM bên NestJS là **Data Mapper** — entity chỉ là dữ liệu thuần, phải nhờ `repository.save(entity)`. Active Record viết nhanh hơn nhiều; đổi lại model gánh cả logic truy vấn nên dễ phình to — vì vậy dự án lớn hay tách bớt sang Service/Query class.

4.5 Factory & Seeder — dữ liệu giả

```
// database/factories/PostFactory.php
public function definition(): array
{
    return [
        'title' => fake()->sentence(),
        'body' => fake()->paragraphs(3, true),
    ];
}

// Dùng ở seeder / test / tinker
Post::factory()->count(30)->create();
```

Việc cần làm:

1. Tạo model + migration + factory: `php artisan make:model Post -mf`
2. Viết schema bảng posts (title, body, timestamps), chạy `php artisan migrate`
3. Khai báo fillable + casts() cho Post
4. Viết PostFactory, seed 30 bản ghi, kiểm tra bằng `php artisan tinker: gõ Post::count()`
5. Chuyển toàn bộ 5 action trong Controller sang dùng Eloquent thật (dùng paginate cho index)
6. Thử bỏ field ra khỏi fillable rồi gọi `create()` — quan sát field đó bị âm thầm bỏ qua như thế nào

Validation với Form Request

Mục tiêu: Đảm bảo dữ liệu client gửi lên luôn đúng định dạng trước khi chạm tới Controller — và trả lỗi theo đúng chuẩn **422 Unprocessable Content**.

5.1 Hai cách validate

```
// Cách 1 – validate ngay trong controller (nhanh, cho case đơn giản)
public function store(Request $request)
{
    $data = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'body'  => ['required', 'string'],
    ]);

    return Post::create($data);
}
```

```
// Cách 2 – Form Request (KHUYẾN NGHỊ cho API): tách rule ra class riêng
php artisan make:request StorePostRequest
```

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // false → tự trả 403 Forbidden
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255', 'unique:posts,title'],
            'body'  => ['required', 'string', 'min:10'],
        ];
    }

    public function messages(): array
    {
        return ['title.required' => 'Tiêu đề là bắt buộc.'];
    }
}
```

5.2 Cơ chế: nó chạy lúc nào?

Bạn chỉ cần type-hint Form Request vào controller. Container thấy class đó kế thừa FormRequest nên tự chạy validate ngay khi resolve tham số — tức là trước khi dòng code đầu tiên trong thân method chạy:

```
public function store(StorePostRequest $request)
{
    // Tôi được đây = dữ liệu CHẮC CHẮN đã hợp lệ
    return Post::create($request->validated());
}
```

Thứ tự thực tế: `authorize()` → `rules()` → (sai thì 422 / không có quyền thì 403) → thân method.

5.3 Format lỗi 422 mà Laravel trả về

Nếu request có header `Accept: application/json` (hoặc là XHR), Laravel trả về đúng cấu trúc sau — hãy nhớ nó vì frontend sẽ dựa vào đây:

```
HTTP/1.1 422 Unprocessable Content

{
  "message": "The title field is required. (and 1 more error)",
  "errors": {
    "title": ["The title field is required."],
    "body": ["The body field must be at least 10 characters."]
  }
}
```

Bẫy kinh điển: nếu **không** gửi header `Accept: application/json`, Laravel coi đây là request từ browser nên trả về **redirect 302** kèm lỗi trong session thay vì JSON 422. Rất nhiều người tưởng API bị lỗi trong khi chỉ là thiếu header.

5.4 Lấy dữ liệu đã validate

Cách gọi	Trả về
<code>\$request->validated()</code>	Mảng chỉ chứa các field có trong rules — field lạ bị loại bỏ
<code>\$request->safe()->only([...])</code>	Chỉ lấy một số field trong dữ liệu đã validate
<code>\$request->safe()->except([...])</code>	Loại một số field
<code>\$request->all()</code>	Toàn bộ input thô , chưa lọc — không nên đưa thẳng vào <code>create()</code>

Đối chiếu NestJS: `validated()` chính là tương đương của `whitelist: true` trong `ValidationPipe` — chỉ nhận field đã khai báo. Còn muốn giống `forbidNonWhitelisted: true` (có field lạ thì báo lỗi luôn), bên Laravel bạn thêm rule `'field_lạ' => 'prohibited'` hoặc kiểm tra trong `after()`.

5.5 Rule hay dùng cho API

Rule	Ý nghĩa
<code>required</code> / <code>nullable</code>	Bắt buộc có / cho phép null
<code>sometimes</code>	Chỉ validate NẾU field xuất hiện — dùng cho PATCH / UpdateRequest
<code>string</code> , <code>integer</code> , <code>boolean</code> , <code>array</code>	Kiểm tra kiểu
<code>max:255</code> , <code>min:10</code> , <code>between:1,10</code>	Giới hạn độ dài / giá trị
<code>email</code> , <code>url</code> , <code>date</code> , <code>uuid</code>	Định dạng
<code>unique:posts,title</code>	Không trùng trong bảng. Khi update phải bỏ qua chính bản ghi đó: <code>Rule::unique('posts')->ignore(\$this->post)</code>
<code>exists:posts,id</code>	Phải tồn tại trong bảng — dùng cho khoá ngoại
<code>confirmed</code>	<code>password</code> phải khớp <code>password_confirmation</code>

Việc cần làm:

1. Tạo `StorePostRequest` và `UpdatePostRequest`
2. `UpdatePostRequest` dùng `sometimes` để mọi field đều optional (tương đương `PartialType` bên NestJS)
3. Type-hint chúng vào `store()` / `update()`, dùng `validated()` khi tạo/cập nhật
4. Test: gửi body thiếu field → xác nhận nhận đúng **422** với cấu trúc `{message, errors}`
5. Test: gửi body thiếu header `Accept: application/json` → quan sát response 302 để hiểu cái bẫy ở mục 5.3
6. Thêm rule `unique` cho `title`, xử lý case update chính bản ghi đó mà không bị báo trùng

API Resources — tầng transform response

Mục tiêu: Không bao giờ trả model thô ra ngoài. Có một tầng riêng quyết định "API này công khai những field nào, định dạng ra sao".

6.1 Vì sao không return thẳng model?

Trả `return $post;` thì Laravel tự serialize toàn bộ cột trong bảng thành JSON. Ba vấn đề: (1) **rò rỉ dữ liệu** — thêm cột `internal_note` vào DB là API tự động phơi ra; (2) **API vỡ hợp đồng** — đổi tên cột trong DB làm frontend chết theo; (3) không kiểm soát được định dạng (ngày tháng, field tính toán). API Resource giải quyết cả ba.

```
php artisan make:resource PostResource
```

```
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class PostResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'          => $this->id,
            'title'       => $this->title,
            'body'        => $this->body,
            'created_at' => $this->created_at->toIso8601String(),
        ];
    }
}
```

Vì sao gọi được `$this->id` khi `$this` là Resource chứ không phải Post? Vì `JsonResource` proxy mọi lời gọi property/method xuống model bên dưới. Model thật nằm ở `$this->resource`.

6.2 Dùng trong Controller

```
// 1 bản ghi
return new PostResource($post);
return $post->toResource(); // cách gọn (Laravel 12+), tự tìm PostResource

// nhiều bản ghi
return PostResource::collection(Post::all());
return Post::latest()->paginate(15)->toResourceCollection();

// kèm status code & header
return (new PostResource($post))->response()->setStatusCode(201);
```

6.3 Data wrapping — response trông thế nào

Mặc định Laravel bọc kết quả trong key `data`:

```
{ "data": { "id": 1, "title": "Hello" } }
```

Khi phân trang, response luôn có thêm `links` và `meta`:

```
{
  "data": [ ... ],
  "links": { "first": "...?page=1", "last": "...?page=4", "prev": null, "next": "...?page=2" },
  "meta": { "current_page": 1, "from": 1, "last_page": 4, "per_page": 15, "to": 15, "total": 54 }
}
```

Muốn bỏ lớp data, gọi `JsonResource::withoutWrapping()` trong `AppServiceProvider::boot()`. Lưu ý: response phân trang **vẫn giữ** data vì cần chỗ đặt meta/links.

Khuyến nghị: giữ nguyên wrapping mặc định. Một API mà "khi thì trả object, khi thì trả object bọc data" là nguồn bug bất tiện cho frontend. Chọn một kiểu và giữ nhất quán toàn bộ endpoint.

6.4 Field có điều kiện

```
return [
    'id'      => $this->id,
    'title'   => $this->title,

    // chỉ trả khi thoả điều kiện
    'secret'  => $this->when($request->user()?->isAdmin(), $this->secret),

    // chỉ trả khi relation Đã được eager load → tránh N+1 (xem Part 08)
    'comments' => CommentResource::collection($this->whenLoaded('comments')),

    // chỉ trả khi đã loadCount('comments')
    'comments_count' => $this->whenCounted('comments'),
];
```

6.5 JSON:API Resource (mới ở Laravel 13)

Laravel 13 bổ sung `JsonApiResource` — tạo response đúng chuẩn jsonapi.org: có type, attributes, relationships, hỗ trợ `?include=` và sparse fieldsets `?fields[posts]=title`, tự set header `application/vnd.api+json`.

```
php artisan make:resource PostResource --json-api
```

```
class PostResource extends JsonApiResource
{
    public $attributes = ['title', 'body', 'created_at'];

    public $relationships = ['author', 'comments'];
}
```

Có nên dùng JSON:API ngay? Với crash course này thì **không cần** — hãy làm JsonResource thường cho vững đã. Biết là nó tồn tại, để khi gặp dự án yêu cầu chuẩn JSON:API thì không phải tự viết tay.

Việc cần làm:

1. Tạo PostResource, chỉ expose: id, title, body, created_at (ISO 8601)
2. Sửa cả 5 action trong Controller để trả về Resource thay vì model thô
3. Endpoint index: dùng paginate (15) + resource collection, kiểm tra response có meta và links
4. Thêm 1 cột "nội bộ" vào bảng posts (vd internal_note), xác nhận nó KHÔNG lộ ra API nhờ Resource
5. Đảm bảo status code đúng: store → 201, destroy → 204

Middleware & Exception Handling

Mục tiêu: Tự viết middleware, và chuẩn hoá **toàn bộ** response lỗi của API về một format duy nhất — đây là thứ phân biệt API "tập viết" với API dùng được thật.

7.1 Viết middleware

```
php artisan make:middleware LogRequests
```

```
public function handle(Request $request, Closure $next): Response
{
    $start = microtime(true);

    $response = $next($request);          // đi sâu vào app

    Log::info('API', [
        'method' => $request->method(),
        'uri'     => $request->path(),
        'status'  => $response->status(),
        'ms'      => round((microtime(true) - $start) * 1000),
    ]);

    return $response;                    // PHẢI return, nếu không request bị treo
}
```

Middleware là **pipeline hai chiều**: code trước `$next()` chạy khi request đi vào, code sau `$next()` chạy khi response đi ra. Nó gánh vai trò của cả **Middleware + Guard + Interceptor** bên NestJS.

7.2 Đăng ký middleware

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
    $middleware->append(LogRequests::class);          // global, chạy cho mọi request

    $middleware->api(prepend: ['throttle:api']);      // thêm vào nhóm api

    $middleware->alias(['admin' => EnsureUserIsAdmin::class]); // đặt tên ngắn
})
```

Sau khi có alias, gắn vào route: `Route::delete(...)->middleware('admin')`. Middleware nhận tham số qua dấu : — ví dụ `'role:editor,publisher'` tương ứng `handle($request, $next, ...$roles)`.

Nhóm middleware **api** ở **Laravel 13** rất mỏng — mặc định chỉ có `SubstituteBindings`. Nghĩa là **rate limit KHÔNG bật sẵn**; muốn có thì tự thêm `'throttle:api'` như trên (chi tiết ở Part 10).

7.3 Exception Handling — chuẩn hoá lỗi JSON

Mặc định Laravel đã trả lỗi dạng JSON khi request có `Accept: application/json: 404` cho model không tìm thấy, 422 cho validation, 401 khi chưa đăng nhập. Nhưng format mỗi loại một kiểu. Muốn thống nhất, cấu

hình trong `withExceptions()`:

```
// bootstrap/app.php
->withExceptions(function (Exceptions $exceptions) {

    // 1) Bắt mọi request /api/* đều được coi là muốn nhận JSON
    $exceptions->shouldRenderJsonWhen(fn (Request $request, Throwable $e) =>
        $request->is('api/*') || $request->expectsJson());

    // 2) Đổi thông điệp 404 cho API (mặc định message rỗng, khó debug)
    $exceptions->render(function (NotFoundHttpException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json(['message' => 'Resource not found.'], 404);
        }
    });
});
```

Bài học từ audit dự án NestJS của bạn: khi tự viết filter/handler lỗi, tuyệt đối **đừng bọc lồng** payload lỗi. Nếu response thành `{"message": {"message": "...", "statusCode": 404}}` thì client phải đọc `body.message.message` — sai chuẩn và rất dễ gây bug. Format lỗi nên **phẳng** và **giống nhau ở mọi status code**, ví dụ: `{ message, errors?, status }`.

7.4 Exception nghiệp vụ tự viết

```
php artisan make:exception PostAlreadyPublishedException
```

```
class PostAlreadyPublishedException extends Exception
{
    // Có method render() → Laravel tự gọi, không cần khai gì trong bootstrap/app.php
    public function render(Request $request): JsonResponse
    {
        return response()->json([
            'message' => 'Bài viết này đã được publish trước đó.',
        ], 409); // 409 Conflict
    }
}
```

Trong service chỉ cần `throw new PostAlreadyPublishedException();` — không cần try/catch ở controller. Ngoài `render()`, exception còn có thể khai `report()` để tự quyết cách ghi log, và `context()` để thêm dữ liệu vào log.

7.5 Các exception → status code Laravel tự map

Exception	Status	Sinh ra khi
<code>ValidationException</code>	422	Form Request / <code>validate()</code> thất bại
<code>ModelNotFoundException</code>	404	<code>findOrFail()</code> , route model binding không thấy
<code>NotFoundHttpException</code>	404	Không route nào khớp
<code>AuthenticationException</code>	401	Middleware <code>auth</code> chặn
<code>AuthorizationException</code>	403	<code>authorize()</code> trả false / policy từ chối
<code>MethodNotAllowedHttpException</code>	405	Đúng URI nhưng sai HTTP verb
<code>ThrottleRequestsException</code>	429	Vượt rate limit

Việc cần làm:

1. Viết LogRequests middleware (log method + uri + status + thời gian xử lý), đăng ký global
2. Viết 1 middleware có tham số (vd EnsureHeader:X-Client-Version), đặt alias và gắn lên vài route
3. Cấu hình withExceptions(): dùng shouldRenderJsonWhen cho api/* và chuẩn hoá message 404
4. Tự viết 1 exception nghiệp vụ có render() trả 409, ném từ service và test
5. Gọi thử đủ 7 tình huống trong bảng 7.5 bằng curl, ghi lại status + body thực tế nhận được
6. Đặt APP_DEBUG=false rồi gây lỗi 500 — so sánh response với lúc true để thấy vì sao production bắt buộc false

Quan hệ dữ liệu (Relationships) & vấn đề N+1

Mục tiêu: Thêm entity Comment liên kết với Post, và hiểu N+1 query — lỗi hiệu năng phổ biến nhất của mọi ORM.

8.1 Khai báo quan hệ 1-N

```
// migration của bảng comments
$table->foreignId('post_id')->constrained()->cascadeOnDelete();
// constrained(): tự suy ra bảng posts + tạo khoá ngoại
// cascadeOnDelete(): xoá post thì comment tự xoá theo
```

```
// app/Models/Post.php — một Post có NHIỀU Comment
public function comments(): HasMany
{
    return $this->hasMany(Comment::class);
}

// app/Models/Comment.php — một Comment thuộc về MỘT Post
public function post(): BelongsTo
{
    return $this->belongsTo(Post::class);
}
```

Quan hệ	Khoá ngoại nằm ở	Ví dụ
hasMany	Bảng con	Post → nhiều Comment
belongsTo	Chính bảng này	Comment → một Post
hasOne	Bảng con	User → một Profile
belongsToMany	Bảng trung gian (pivot)	Post ↔ Tag

8.2 Truy vấn quan hệ

```
$post->comments; // property → chạy query LÚC gọi, trả Collection
$post->comments(); // method → trả query builder, còn nối được điều kiện

$post->comments()->where('is_approved', true)->latest()->get();

$post->comments()->create(['body' => 'Hay quá!']); // tự gán post_id
```

8.3 N+1 — lỗi hiệu năng phải hiểu

```
// ✗ SAI — sinh 1 + N query
$post = Post::all(); // 1 query lấy 30 post
foreach ($posts as $post) {
    echo $post->comments->count(); // + 30 query nữa (mỗi post 1 query)
}
// → tổng 31 query cho một trang danh sách

// ✓ ĐÚNG — eager loading, chỉ 2 query
$post = Post::with('comments')->get();
// query 1: select * from posts
// query 2: select * from comments where post_id in (1,2,3,...)

// ✓ Chỉ cần ĐẾM thì đừng load hết dữ liệu
$post = Post::withCount('comments')->get(); // có $post->comments_count
```

Bắt N+1 tự động khi dev: thêm vào `AppServiceProvider::boot()`:

```
Model::preventLazyLoading(! $this->app->isProduction());
```

Từ đó bất kỳ chỗ nào quên `with()` sẽ **throw exception ngay ở local** thay vì âm thầm chạy chậm ở production.

Đây là một trong những setting đáng bật nhất của Laravel.

Ghép với Part 06: Resource dùng `whenLoaded('comments')` — nếu controller quên `with('comments')` thì field đó đơn giản không xuất hiện, chứ không âm thầm bắn thêm N query. Hai tính năng này thiết kế để đi cùng nhau.

8.4 Eager load có điều kiện & lồng nhau

```
Post::with(['comments' => fn ($q) => $q->where('is_approved', true)])->get();

Post::with('comments.author')->get(); // lồng 2 cấp

$post->load('comments'); // load sau khi đã có model

Post::has('comments')->get(); // chỉ post CÓ comment
Post::doesntHave('comments')->get(); // chỉ post KHÔNG có comment
```

Việc cần làm:

1. Tạo model + migration Comment (post_id, body), khai `foreignId()->constrained()->cascadeOnDelete()`
2. Khai `hasMany / belongsTo` ở hai model
3. Endpoint GET `/api/posts/{post}` trả post kèm comments (dùng `with()` + `whenLoaded()`)
4. Endpoint lồng: GET `/api/posts/{post}/comments` và POST `/api/posts/{post}/comments`
5. Bật `Model::preventLazyLoading()`, cố tình bỏ `with()` để thấy exception
6. Dùng `withCount('comments')` cho endpoint index và trả qua `whenCounted()`
7. Cài Laravel Telescope hoặc dùng `DB::listen()` để đếm số query thực tế của endpoint index — chứng minh đã hết N+1

Authentication với Laravel Sanctum

Mục tiêu: Hiểu trọn luồng đăng ký → đăng nhập → cấp token → bảo vệ route → đăng xuất, và biết vì sao password phải hash chứ không được mã hoá.

9.1 Sanctum là gì — và không phải là gì

Sanctum đã được cài sẵn khi bạn chạy `php artisan install:api` ở Part 01. Nó giải quyết 2 bài toán tách biệt:

- **API token** (personal access token, giống token của GitHub): token là chuỗi random, được **hash SHA-256** rồi lưu vào bảng `personal_access_tokens`. Client gửi kèm header `Authorization: Bearer <token>`.
→ Đây là phần ta dùng.
- **SPA authentication**: dùng session cookie thay vì token, cho frontend cùng domain. Không dùng trong crash course này.

Sanctum khác JWT thế nào? Token Sanctum là **opaque** (chuỗi vô nghĩa, phải tra DB mới biết của ai) — nên **thu hồi được ngay lập tức** bằng cách xoá dòng trong DB. JWT là **self-contained** (chứa payload, verify bằng chữ ký, không cần tra DB) — nhanh hơn nhưng **không thu hồi được** trước khi hết hạn. Với API thường, Sanctum là lựa chọn mặc định đúng. Cần OAuth2 đầy đủ thì mới dùng Laravel Passport.

9.2 Chuẩn bị model User

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;    // thêm HasApiTokens
}
```

Trait `HasApiTokens` mang lại: `createToken()`, `tokens()`, `currentAccessToken()`, `tokenCan()`.

9.3 Luồng đầy đủ

1. POST /api/register

Validate → **hash password** → tạo user → (tùy chọn) cấp token luôn

2. POST /api/login

Tìm user theo email → Hash::check() so password → khớp thì createToken() → trả plainTextToken

3. Client lưu token

Mọi request sau gửi header Authorization: Bearer <token>

4. auth:sanctum

Middleware hash token nhận được, tra bảng personal_access_tokens → hợp lệ thì gán user vào \$request->user(); sai/hết hạn → **401**

5. POST /api/logout

\$request->user()->currentAccessToken()->delete() → token đó chết ngay

9.4 Code các endpoint

```
// Đăng nhập – cách docs Laravel khuyến nghị
public function login(LoginRequest $request)
{
    $user = User::where('email', $request->email)->first();

    if (!$user || !Hash::check($request->password, $user->password)) {
        throw ValidationException::withMessages([
            'email' => ['The provided credentials are incorrect.'],
        ]); // → 422 với format lỗi chuẩn
    }

    return [
        'token' => $user->createToken($request->device_name)->plainTextToken,
    ];
}
```

Vì sao báo lỗi chung **"credentials are incorrect"** thay vì **"email không tồn tại"**? Nếu phân biệt hai thông báo, kẻ tấn công có thể dò xem email nào đã đăng ký (user enumeration). Luôn trả cùng một message cho cả hai trường hợp.

```
// routes/api.php
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', fn (Request $r) => $r->user());
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('posts', PostController::class)->except(['index', 'show']);
});
```

9.5 Hash password — nguyên lý

Laravel dùng **bcrypt** (mặc định) qua `Hash::make($password)`. Đặc điểm:

- **Một chiều** — không có hàm nào "giải mã" hash về lại password. Khi login ta **hash lại rồi so 2 hash** (Hash: :check()), chứ không giải mã.
- **Có salt ngẫu nhiên** — cùng một password hash 2 lần cho 2 chuỗi khác nhau ⇒ chống rainbow table.
- **Chậm có chủ đích** — cost factor cao làm brute-force tốn kém.

```
// Cách gọn nhất: dùng cast 'hashed' — Laravel tự hash khi gán
protected function casts(): array
{
    return ['password' => 'hashed']; // User model mặc định đã có sẵn dòng này
}

// Nhờ vậy chỉ cần:
User::create(['password' => $request->password]); // tự hash, KHÔNG lưu plain text
```

9.6 Token abilities, hết hạn, thu hồi

```
// Cấp token kèm quyền (giống scope của OAuth)
$user->createToken('mobile', ['post:create', 'post:update'])->plainTextToken;

// Kiểm tra quyền trong code
if ($request->user()->tokenCan('post:create')) { /* ... */ }

// Hết hạn: config/sanctum.php → 'expiration' => 525600 (phút) — mặc định null = không hết hạn
// Dọn token hết hạn định kỳ:
Schedule::command('sanctum:prune-expired --hours=24')->daily();

// Thu hồi
$request->user()->currentAccessToken()->delete(); // logout thiết bị hiện tại
$user->tokens()->delete(); // logout mọi thiết bị
```

9.7 Phân quyền: Policy

Authentication = "bạn là ai". Authorization = "bạn có được làm việc này không". Việc thứ hai dùng Policy:

```
php artisan make:policy PostPolicy --model=Post
```

```
public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id; // chỉ tác giả được sửa
}
```

```
// Trong controller
$this->authorize('update', $post); // không có quyền → 403

// Hoặc dùng attribute (Laravel 13)
#[Authorize('update', 'post')]
public function update(UpdatePostRequest $request, Post $post) { /* ... */ }
```

Việc cần làm:

1. Thêm trait HasApiTokens vào model User, chạy migration cho bảng `personal_access_tokens`
2. Viết AuthController với 3 action: register, login, logout (+ route /me)
3. Tạo RegisterRequest / LoginRequest (email unique:users, password confirmed, min:8)
4. Bọc các route ghi (store/update/destroy) trong auth:sanctum
5. Test bằng curl: gọi POST /api/posts không token → phải nhận **401**; có token → 201
6. Mở bảng `personal_access_tokens` trong DB, xác nhận cột token là hash chứ không phải chuỗi bạn nhận được
7. Thêm cột `user_id` vào posts + PostPolicy: chỉ tác giả được sửa/xoá. Test bằng 2 user khác nhau → user B sửa post của A phải nhận **403**

Chuẩn hoá REST API cho production

Mục tiêu: Biến API "chạy được" thành API "dùng được thật": status code đúng, phân trang / filter / sort, versioning, rate limit, CORS, và checklist trước khi deploy.

10.1 Status code — dùng đúng, đừng trả 200 cho mọi thứ

Code	Tên	Dùng khi
200	OK	GET thành công, PUT/PATCH thành công (trả bản ghi sau khi sửa)
201	Created	POST tạo mới thành công — nên kèm header <code>Location</code>
204	No Content	DELETE thành công, không có body trả về
400	Bad Request	Request hỏng về mặt cú pháp (JSON sai định dạng)
401	Unauthorized	Chưa đăng nhập / token sai (thực chất là "unauthenticated")
403	Forbidden	Đã đăng nhập nhưng không có quyền
404	Not Found	Không tồn tại resource
409	Conflict	Xung đột trạng thái (publish 2 lần, trùng dữ liệu)
422	Unprocessable Content	Cú pháp đúng nhưng dữ liệu không hợp lệ (validation)
429	Too Many Requests	Vượt rate limit
500	Server Error	Lỗi phía server — không bao giờ được lộ stack trace ra client

Nhớ mẹo phân biệt 401 vs 403: 401 = "tôi không biết bạn là ai" (thiếu/sai token → client nên đi đăng nhập lại). 403 = "tôi biết bạn là ai, nhưng bạn không được phép" (đăng nhập lại vô ích).

10.2 Phân trang, filter, sort

```
// GET /api/posts?page=2&per_page=20&search=nest&sort=-created_at
public function index(Request $request)
{
    $perPage = min($request->integer('per_page', 15), 100); // CHẶN TRẦN

    $posts = Post::query()
        ->when($request->filled('search'), fn ($q) =>
            $q->where('title', 'like', '%'.$request->string('search').'%'))
        ->withCount('comments')
        ->latest()
        ->paginate($perPage)
        ->withQueryString(); // giữ lại query param trong link phân trang

    return PostResource::collection($posts);
}
```

Ba lỗi bảo mật/hiệu năng hay gặp ở endpoint list: (1) không chặn trần per_page → client gọi ? per_page=999999 làm sập DB; (2) cho sort theo tên cột client tự gửi mà không whitelist → SQL injection qua orderBy; (3) trả Post::all() không phân trang.

```
// Sort an toàn: LUÔN whitelist tên cột
$sortable = ['created_at', 'title'];
$sort = $request->string('sort', '-created_at');
$dir = str_starts_with($sort, '-') ? 'desc' : 'asc';
$col = ltrim($sort, '-');

if (in_array($col, $sortable, true)) {
    $query->orderBy($col, $dir);
}
```

10.3 Rate limiting

Nhớ Part 07: nhóm api của Laravel 13 **không** bật throttle sẵn. Làm 2 bước:

```
// Bước 1 – định nghĩa limiter trong AppServiceProvider::boot()
RateLimiter::for('api', fn (Request $request) =>
    Limit::perMinute(60)->by($request->user()?->id ?: $request->ip()));

// Giới hạn chặt hơn cho login (chống brute-force)
RateLimiter::for('login', fn (Request $request) => [
    Limit::perMinute(5)->by($request->input('email')),
    Limit::perMinute(20)->by($request->ip()),
]);
```

```
// Bước 2 – gắn vào route hoặc nhóm api
$middleware->api(prepend: ['throttle:api']); // bootstrap/app.php

Route::post('/login', ...)->middleware('throttle:login'); // routes/api.php
```

Vượt hạn mức → Laravel tự trả **429** kèm header Retry-After và X-RateLimit-Remaining.

10.4 CORS

Middleware **HandleCors** đã nằm trong stack global sẵn. Chỉ khi cần tùy biến mới publish config:

```
php artisan config:publish cors # → tạo config/cors.php
```

Trong đó khai allowed_origins (đừng để ['*'] ở production nếu API có auth bằng cookie), allowed_headers, supports_credentials.

10.5 API versioning

```
// Cách 1: prefix trong routes/api.php
Route::prefix('v1')->group(function () {
    Route::apiResource('posts', V1\PostController::class);
}); // → /api/v1/posts

// Cách 2: đổi prefix mặc định trong bootstrap/app.php
->withRouting(api: __DIR__.'/../routes/api.php', apiPrefix: 'api/v1')
```

Nguyên tắc: khi đã public API, **không phá vỡ hợp đồng cũ** — đổi tên/xoá field là breaking change, phải ra v2 chứ không sửa v1.

10.6 Checklist trước khi deploy

Việc	Vì sao
<code>APP_DEBUG=false</code>	Quan trọng nhất. True ở production = lộ config, DB credential, stack trace
<code>APP_ENV=production</code>	Bật các tối ưu & tắt hành vi dev
<code>php artisan key:generate</code>	Phải có APP_KEY riêng, không dùng chung với repo
<code>php artisan config:cache</code> <code>php artisan route:cache</code>	Tăng tốc rõ rệt. Nhớ: sau khi cache thì <code>env()</code> ngoài config trả null
<code>composer install --no-dev --optimize-autoloader</code>	Bỏ package dev, tối ưu autoload
Document root trở vào <code>public/</code>	Trở sai = phơi <code>.env</code> ra internet
Bật HTTPS + rate limit	Token gửi qua HTTP là token bị lộ
Kiểm tra <code>storage/</code> ghi được	Không ghi được log/cache = lỗi 500 bí ẩn

Việc cần làm:

1. Rà lại toàn bộ endpoint: status code đúng chuẩn bảng 10.1 chưa (đặc biệt 201 và 204)
2. Endpoint index: thêm `per_page` (có chặn trần), `search`, `sort` có whitelist
3. Định nghĩa limiter api và login, gắn throttle, test bằng cách gọi liên tục cho tới khi nhận **429**
4. Thêm prefix version v1 cho toàn bộ route
5. Chạy `config:cache` + `route:cache` ở local, xác nhận app vẫn chạy, rồi `optimize:clear`
6. Viết file `.env.example` đầy đủ và một mục "API endpoints" trong README (method, path, body, response mẫu)

Testing (Pest / PHPUnit)

Mục tiêu: Viết test gọi thẳng vào API và kiểm tra response — cách nhanh nhất để không bao giờ phải mở Postman test tay lại lần thứ hai.

11.1 Pest hay PHPUnit?

Laravel 11+ mặc định dùng **Pest** (cú pháp function, gọn), nhưng vẫn chạy trên PHPUnit bên dưới nên mọi assertion đều giống nhau. Chọn kiểu nào cũng được — **đừng trộn 2 kiểu trong cùng dự án**.

```
php artisan make:test PostApiTest           # feature test (mặc định)
php artisan make:test PostServiceTest --unit # unit test
php artisan test                             # chạy toàn bộ
php artisan test --filter=PostApiTest        # chạy 1 file
```

11.2 Feature test cho API

```
// tests/Feature/PostApiTest.php (Pest)
use App\Models\{Post, User};
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Sanctum\Sanctum;

uses(RefreshDatabase::class); // mỗi test chạy trên DB sạch, tự rollback sau đó

test('ai cũng xem được danh sách post', function () {
    Post::factory()->count(3)->create();

    $this->getJson('/api/v1/posts')
        ->assertOk()
        ->assertJsonCount(3, 'data')
        ->assertJsonStructure(['data' => [['id', 'title', 'body']], 'meta', 'links']);
});

test('chưa đăng nhập thì không tạo được post', function () {
    $this->postJson('/api/v1/posts', ['title' => 'X', 'body' => 'Y'])
        ->assertUnauthorized(); // 401
});

test('validate báo lỗi 422 đúng field', function () {
    Sanctum::actingAs(User::factory()->create());

    $this->postJson('/api/v1/posts', ['title' => ''])
        ->assertStatus(422)
        ->assertJsonValidationErrors(['title', 'body']);
});

test('tác giả tạo post thành công', function () {
    Sanctum::actingAs($user = User::factory()->create());

    $this->postJson('/api/v1/posts', ['title' => 'Hello', 'body' => 'Nội dung đủ dài'])
        ->assertCreated() // 201
        ->assertJsonPath('data.title', 'Hello');

    $this->assertDatabaseHas('posts', ['title' => 'Hello', 'user_id' => $user->id]);
});
```

11.3 Assertion hay dùng

Assertion	Kiểm tra
<code>assertOk()</code> / <code>assertStatus(n)</code>	200 / status bất kỳ
<code>assertCreated()</code> / <code>assertNoContent()</code>	201 / 204
<code>assertUnauthorized()</code> / <code>assertForbidden()</code> / <code>assertNotFound()</code>	401 / 403 / 404
<code>assertJson([...])</code>	JSON chứa đoạn này (không cần khớp hết)
<code>assertExactJson([...])</code>	JSON khớp chính xác toàn bộ
<code>assertJsonStructure([...])</code>	Đúng "hình dạng" key, không quan tâm giá trị
<code>assertJsonPath('data.title', 'X')</code>	Giá trị tại một đường dẫn cụ thể
<code>assertJsonCount(3, 'data')</code>	Số phần tử trong mảng
<code>assertJsonValidationErrors([...])</code>	Có lỗi validation ở đúng các field này
<code>assertDatabaseHas</code> / <code>Missing</code> / <code>Count</code>	Trạng thái thật trong database

RefreshDatabase làm gì: bọc mỗi test trong một transaction và rollback khi test kết thúc → các test độc lập, không rác dữ liệu. Cấu hình DB test riêng trong `phpunit.xml` (thường dùng SQLite in-memory: `DB_DATABASE=:memory:`) để chạy cực nhanh và không đụng vào DB dev.

Việc cần làm (tùy chọn, làm sau khi xong Part 1–10):

1. Cấu hình `phpunit.xml` dùng SQLite in-memory cho test
2. Viết feature test cho đủ 5 endpoint của posts (index, show, store, update, destroy)
3. Viết test cho 3 case lỗi: 401 (không token), 403 (không phải tác giả), 422 (validate sai)
4. Viết 1 test chứng minh không còn N+1: dùng `DB::listen()` đếm query của endpoint index
5. Chạy `php artisan test` — mục tiêu: toàn bộ xanh

— Hết tài liệu — Quay lại chat khi bạn xong mỗi Part để được review & giải thích tiếp phần sau —